rqbit 파일 스트리밍 구현 분석

개요

이 문서는 rqbit BitTorrent 클라이언트의 파일 스트리밍 기능을 분석합니다. rqbit은 HTTP Range 요청을 지원하여 부분 콘텐츠 서빙을 제공하고, 스트리밍 중 조각 우선순위를 동적으로 조정하여 실시간 스트리밍과 시크 기능을 지원합니다.

HTTP 범위 요청 및 부분 콘텐츠 서빙

1. 스트리밍 핸들러 구조

StreamPathParams 구조체

```
#[derive(Deserialize)]

pub struct StreamPathParams {

id: TorrentIdOrHash,

file_id: usize,

#[serde(rename = "filename")]

_filename: Option<Arc<str>>,

}
```

URL 경로에서 토렌트 ID, 파일 ID, 그리고 선택적 파일명을 추출합니다. 파일명은 SEO와 사용자 편의를 위해 포함되지만 실제 처리에는 사용되지 않습니다.

메인 스트리밍 핸들러

```rust
pub async fn h_torrent_stream_file(
State(state): State<ApiState>,
Path(StreamPathParams { id, file_id, .. }): Path<StreamPathParams>,
headers: http::HeaderMap,
) -> Result<impl IntoResponse> {
let mut stream = state.api.api_stream(id, file_id)?;
let mut status = StatusCode::OK;
let mut output_headers = HeaderMap::new();
output_headers.insert("Accept-Ranges", HeaderValue::from_static("bytes"));
// Range 헤더 처리 로직...
}
```

2. HTTP Range 요청 처리

Range 헤더 파싱

```rust
let range_header = headers.get(http::header::RANGE);
if let Some(range) = range_header {
let offset: Option<u64> = range
.to_str()
.ok()
.and_then(|s| s.strip_prefix("bytes="))
.and_then(|s| s.strip_suffix('-'))
.and_then(|s| s.parse().ok());
```

```
if let Some(offset) = offset {
status = StatusCode::PARTIAL_CONTENT;
stream.seek(SeekFrom::Start(offset)).await.context("error seeking")?;
// Content-Length와 Content-Range 헤더 설정
output_headers.insert(
http::header::CONTENT_LENGTH,
HeaderValue::from_str(&format!("{}", stream.len() - stream.position()))?
);
output_headers.insert(
http::header::CONTENT_RANGE,
HeaderValue::from_str(&format!(
"bytes {}-{}/{}",
stream.position(),
stream.len().saturating_sub(1),
stream.len()
))?
);
}
}
```

특징:

- **단순한 Range 형식 지원**: `bytes=offset-` 형태만 지원 (끝 위치 지정 없음)
- **206 Partial Content**: Range 요청 시 적절한 HTTP 상태 코드 반환

- **Content-Range 헤더**: 클라이언트가 전체 파일 크기와 현재 범위를 알 수 있도록 함
- **비동기 시킹**: `AsyncSeek` 트레이트를 통한 효율적인 파일 위치 이동

3. DLNA 프로토콜 지원

DLNA 헤더 처리

```
const DLNA_TRANSFER_MODE: &str = "transferMode.dlna.org";
const DLNA_GET_CONTENT_FEATURES: &str = "getcontentFeatures.dlna.org";
const DLNA_CONTENT_FEATURES: &str = "contentFeatures.dlna.org";


if headers
.get(DLNA_TRANSFER_MODE)
.map(|v| matches!(v.as_bytes(), b"Streaming" | b"streaming"))
.unwrap_or(false)
{
output_headers.insert(DLNA_TRANSFER_MODE, HeaderValue::from_static("Streaming"))
}


if headers
.get(DLNA_GET_CONTENT_FEATURES)
.map(|v| v.as_bytes() == b"1")
.unwrap_or(false)
{
output_headers.insert(
```

```
DLNA_CONTENT_FEATURES,
HeaderValue::from_static("DLNA.ORG_OP=01"),
);
}
```

특징:

- **DLNA 호환성**: 스마트 TV와 미디어 플레이어 지원
- **스트리밍 모드**: 실시간 스트리밍 최적화
- **콘텐츠 기능 광고**: 시크 지원 등의 기능을 클라이언트에 알림

4. MIME 타입 자동 감지

파일 확장자 기반 MIME 타입 결정

```
fn torrent_file_mime_type(
info: &ValidatedTorrentMetaV1Info<ByteBufOwned>,
file_idx: usize,
) -> Result<&'static str> {
Ok(info
.iter_file_details()
.nth(file_idx)
.and_then(|d| {
d.filename
.iter_components()
.last()
```

```
.and_then(|s| mime_guess::from_path(&*s).first_raw())
})
.ok_or((
StatusCode::INTERNAL_SERVER_ERROR,
"cannot determine mime type for file",
))?)
}
```

특징:

- **mime_guess 크레이트 활용**: 파일 확장자를 통한 MIME 타입 추론
- **파일명 경로 분석**: 다중 경로 구성 요소에서 실제 파일명 추출
- **오류 처리**: MIME 타입을 결정할 수 없는 경우 적절한 오류 반환

FileStream 구현 분석

1. FileStream 구조체

```
pub struct FileStream {
torrent: ManagedTorrentHandle,
metadata: Arc<TorrentMetadata>,
streams: Arc<TorrentStreams>,
stream_id: usize,
file_id: usize,
position: u64,
```

```
// file params
file_len: u64,
file_torrent_abs_offset: u64,

spawner: BlockingSpawner,
}
```

주요 필드:

- **torrent**: 토렌트 핸들 (데이터 접근용)
- **metadata**: 토렌트 메타데이터 (파일 구조 정보)
- **streams**: 전역 스트림 관리자 (우선순위 조정용)
- **stream_id**: 고유 스트림 식별자
- **position**: 현재 파일 내 위치
- **file_len**: 파일 전체 크기
- **file_torrent_abs_offset**: 토렌트 내에서의 절대 오프셋

2. AsyncRead 구현

비동기 읽기 로직

```
impl AsyncRead for FileStream {
fn poll_read(
mut self: std::pin::Pin<&mut Self>,
cx: &mut std::task::Context<'_>,
```

```
tbuf: &mut tokio::io::ReadBuf<'_>,
) -> Poll<std::io::Result<()>> {
// EOF 체크
if self.position == self.file_len {
return Poll::Ready(Ok(()));
}

// 현재 위치에 해당하는 조각 계산
let current = poll_try_io!(
self.metadata
.lengths()
.compute_current_piece(self.position, self.file_torrent_abs_offset)
.context("invalid position")
);

// 조각 가용성 확인
let have = poll_try_io!(self.torrent.with_chunk_tracker(|ct| {
let have = ct.get_have_pieces().as_slice()[current.id.get() as usize];
if !have {
self.streams.register_waker(self.stream_id, cx.waker().clone());
}
have
}));
```

```
if !have {
return Poll::Pending;
}


// 실제 데이터 읽기
let buf = tbuf.initialize_unfilled();
let file_remaining = self.file_len - self.position;
let bytes_to_read: usize = poll_try_io!(
(buf.len() as u64)
.min(current.piece_remaining as u64)
.min(file_remaining)
.try_into()
);


let buf = &mut buf[..bytes_to_read];
poll_try_io!(poll_try_io!(self.spawner.spawn_block_in_place(|| {
self.torrent.with_storage_and_file(
self.file_id,
|files, _fi| {
files.pread_exact(self.file_id, self.position, buf)?;
Ok::<_, anyhow::Error>(())
},
&self.metadata,
```

```
)
})));


self.as_mut().advance(bytes_to_read as u64);
tbuf.advance(bytes_to_read);


Poll::Ready(Ok(()))
}
}
```

특징:

- **조각 기반 읽기**: BitTorrent 조각 단위로 데이터 읽기
- **비동기 대기**: 필요한 조각이 없으면 Waker 등록 후 Pending 반환
- **경계 처리**: 조각 경계, 파일 경계, 버퍼 크기 모두 고려
- **블로킹 작업 분리**: 실제 디스크 I/O는 별도 스레드에서 실행

3. AsyncSeek 구현

시크 기능

```
impl AsyncSeek for FileStream {
fn start_seek(
mut self: std::pin::Pin<&mut Self>,
position: std::io::SeekFrom,
) -> std::io::Result<()> {
```

```
let end_i64 = map_io_err!(TryInto::<i64>::try_into(self.file_len))?;
let new_pos: i64 = match position {
SeekFrom::Start(s) => map_io_err!(s.try_into())?,
SeekFrom::End(e) => map_io_err!(TryInto::<i64>::try_into(self.file_len))? + e,
SeekFrom::Current(o) => map_io_err!(TryInto::<i64>::try_into(self.position))? +
};

if new_pos < 0 || new_pos > end_i64 {
return Err(std::io::Error::new(
std::io::ErrorKind::InvalidInput,
anyhow::anyhow!("invalid seek"),
));
}

self.as_mut().set_position(map_io_err!(new_pos.try_into())?);
Ok(())
}

fn poll_complete(
self: std::pin::Pin<&mut Self>,
_cx: &mut std::task::Context<'_>,
) -> Poll<std::io::Result<u64>> {
Poll::Ready(Ok(self.position))
```

```
}

}
```

특징:

- **표준 시크 모드 지원**: Start, End, Current 모든 모드 지원
- **범위 검증**: 파일 경계를 벗어나는 시크 방지
- **즉시 완료**: 메모리 내 위치 변경만 수행하므로 즉시 완료
- **타입 안전성**: 정수 오버플로우 방지를 위한 명시적 변환

스트리밍 중 조각 우선순위 조정

1. TorrentStreams 관리자

스트림 상태 추적

```
struct StreamState {

file_id: usize,

file_len: u64,

file_abs_offset: u64,

position: u64,

waker: Option<Waker>,

}
```

```
#[derive(Default)]

pub(crate) struct TorrentStreams {
```

```
next_stream_id: AtomicUsize,

streams: DashMap<StreamId, StreamState>,

}
```

특징:

- **동시 스트림 지원**: 여러 파일을 동시에 스트리밍 가능
- **스레드 안전성**: DashMap을 사용한 동시 접근 지원
- **Waker 관리**: 각 스트림별로 비동기 대기 상태 관리

2. 우선순위 조각 선택

인터리브 알고리즘

```rust
pub(crate) fn iter_next_pieces<'a>(

&'a self,

lengths: &'a Lengths,

) -> impl Iterator<Item = ValidPieceIndex> + 'a {

struct Interleave<I> {

all: VecDeque<I>,

}


impl<I: Iterator<Item = ValidPieceIndex>> Iterator for Interleave<I> {

type Item = ValidPieceIndex;


fn next(&mut self) -> Option<Self::Item> {
```

```
while let Some(mut it) = self.all.pop_front() {
if let Some(piece) = it.next() {
self.all.push_back(it);
return Some(piece);
}
}
None
}
}

let mut all: Vec<_> = self.streams.iter().map(|s| s.queue(lengths)).collect();
// 공정성을 위한 셔플
use rand::seq::SliceRandom;
all.shuffle(&mut rand::rng());

Interleave { all: all.into() }
}
```

특징:

- **라운드 로빈 방식**: 각 스트림에서 번갈아가며 조각 선택
- **공정한 대역폭 분배**: 여러 스트림 간 균등한 우선순위 부여
- **랜덤 셔플**: 결정론적 패턴 방지로 더 공정한 분배

스트림별 큐 생성

```
fn queue<'a>(&self, lengths: &'a Lengths) -> impl Iterator<Item = ValidPieceInde

let start = self.file_abs_offset + self.position;

let end = (start + PER_STREAM_BUF_DEFAULT).min(self.file_abs_offset + self.file_

let dpl = lengths.default_piece_length();

let start_id = (start / dpl as u64).try_into().unwrap();

let end_id = end.div_ceil(dpl as u64).try_into().unwrap();

(start_id..end_id).filter_map(|i| lengths.validate_piece_index(i))

}
```

특징:

- **선행 버퍼링**: 현재 위치에서 32MB 앞까지 미리 다운로드
- **적응적 범위**: 파일 끝을 넘지 않도록 자동 조정
- **조각 경계 정렬**: BitTorrent 조각 경계에 맞춘 범위 계산

3. 메인 조각 선택 통합

우선순위 체인

```
let priority_streamed_pieces = self

.state

.streams

.iter_next_pieces(&self.state.lengths)

.filter(|pid| {

!chunk_tracker.is_piece_have(*pid)
```

```
&& !g.inflight_pieces.contains_key(pid)

});

let natural_order_pieces = chunk_tracker

.iter_queued_pieces(&g.file_priorities, &self.state.metadata.file_infos);



for n in priority_streamed_pieces.chain(natural_order_pieces) {

if bf.get(n.get() as usize).map(|v| *v) == Some(true) {

n_opt = Some(n);

break;

}

}
```

특징:

- **스트리밍 우선순위**: 스트리밍 조각을 일반 조각보다 우선 선택
- **중복 방지**: 이미 보유하거나 다운로드 중인 조각 제외
- **피어 가용성 확인**: 연결된 피어가 해당 조각을 보유한지 확인
- **폴백 메커니즘**: 스트리밍 조각이 없으면 일반 순서로 진행

4. 조각 완료 시 알림

Waker 시스템

```
pub(crate) fn wake_streams_on_piece_completed(

&self,

piece_id: ValidPieceIndex,
```

```
lengths: &Lengths,
) {
for mut w in self.streams.iter_mut() {
if w.value().current_piece(lengths).map(|p| p.id) == Some(piece_id) {
if let Some(waker) = w.value_mut().waker.take() {
trace!(
stream_id = *w.key(),
piece_id = piece_id.get(),
"waking stream"
);
waker.wake();
}
}
}
}
```

특징:

- **즉시 알림**: 조각 다운로드 완료 시 대기 중인 스트림 즉시 깨움
- **선택적 알림**: 해당 조각을 기다리는 스트림만 깨움
- **Waker 소비**: 한 번 사용된 Waker는 제거하여 메모리 누수 방지

실시간 스트리밍 지원

1. 적응적 피어 재연결

필요 시 피어 재연결

```
fn maybe_reconnect_needed_peers_for_file(&self, file_id: usize) -> bool {
// 파일이 완료되었으면 재연결 불필요
if self.is_file_finished(file_id) {
return false;
}
self.with_state(|state| {
if let crate::ManagedTorrentState::Live(l) = &state {
l.reconnect_all_not_needed_peers();
}
});
true
}
```

특징:

- **동적 피어 관리**: 스트리밍 시작 시 필요한 피어들과 재연결
- **완료 파일 최적화**: 이미 완료된 파일은 추가 연결 불필요
- **라이브 상태 확인**: 토렌트가 활성 상태일 때만 재연결 수행

2. 스트림 생명주기 관리

스트림 생성

```
pub fn stream(self: Arc<Self>, file_id: usize) -> anyhow::Result<FileStream> {
```

```
let metadata = self
.metadata
.load_full()
.context("torrent metadata is not resolved")?;
let (fd_len, fd_offset) = self.with_storage_and_file(
file_id,
|_fd, fi| (fi.len, fi.offset_in_torrent),
&metadata,
)?;
let streams = self.streams()?;
let s = FileStream {
stream_id: streams.next_id(),
streams: streams.clone(),
file_id,
position: 0,
file_len: fd_len,
file_torrent_abs_offset: fd_offset,
torrent: self,
spawner: BlockingSpawner::default(),
metadata,
};
s.torrent.maybe_reconnect_needed_peers_for_file(file_id);
streams.streams.insert(s.stream_id, /* ... */);
```

```
Ok(s)
}
```

특징:

- **메타데이터 검증**: 스트리밍 전 토렌트 메타데이터 완전 로드 확인
- **파일 정보 추출**: 파일 크기와 토렌트 내 오프셋 계산
- **고유 ID 할당**: 각 스트림에 고유 식별자 부여
- **자동 등록**: 생성과 동시에 전역 스트림 관리자에 등록

스트림 정리

```
impl Drop for FileStream {
fn drop(&mut self) {
self.streams.drop_stream(self.stream_id);
}
}
```

특징:

- **자동 정리**: 스트림 종료 시 자동으로 관리자에서 제거
- **메모리 누수 방지**: 더 이상 사용되지 않는 스트림 상태 정리
- **우선순위 조정**: 활성 스트림 수 감소로 남은 스트림들의 우선순위 향상

3. 성능 최적화

버퍼링 전략

```
const PER_STREAM_BUF_DEFAULT: u64 = 32 * 1024 * 1024; // 32MB 선행 버퍼
```

- **적절한 버퍼 크기**: 네트워크 지연과 메모리 사용량의 균형
- **연속 재생 보장**: 일반적인 비트레이트에서 끊김 없는 재생
- **시크 대응**: 시크 후 빠른 재생 재개를 위한 충분한 버퍼

스트림 처리 최적화

```
let s = tokio_util::io::ReaderStream::with_capacity(stream, 65536);

Ok((status, (output_headers, axum::body::Body::from_stream(s))))
```

특징:

- **청크 크기 최적화**: 64KB 청크로 네트워크 효율성 향상
- **스트림 기반 응답**: 메모리 사용량 최소화
- **백프레셔 지원**: 클라이언트 처리 속도에 맞춘 데이터 전송

시크(Seek) 기능 구현

1. 즉시 시크

FileStream의 AsyncSeek 구현은 메모리 내 위치만 변경하므로 즉시 완료됩니다:

```
fn start_seek(mut self: std::pin::Pin<&mut Self>, position: std::io::SeekFrom) -

// 위치 계산 및 검증

self.as_mut().set_position(new_pos);

Ok(()) // 즉시 완료
```

```
}
```

2. 시크 후 우선순위 조정

시크 후 새로운 위치의 조각들이 자동으로 우선순위를 받습니다:

```
fn set_position(&mut self, new_pos: u64) {
self.position = new_pos;
self.streams
.streams
.get_mut(&self.stream_id)
.unwrap()
.value_mut()
.position = new_pos; // 전역 상태도 업데이트
}
```

3. 적응적 다운로드

시크 후 새로운 위치부터 32MB 범위의 조각들이 우선 다운로드됩니다.

결론

rqbit의 파일 스트리밍 구현은 다음과 같은 특징을 보여줍니다:

1. **표준 호환성**: HTTP Range 요청과 DLNA 프로토콜 지원
2. **동적 우선순위**: 스트리밍 위치에 따른 실시간 조각 우선순위 조정

3. **다중 스트림 지원**: 여러 파일 동시 스트리밍과 공정한 대역폭 분배

4. **효율적인 시크**: 즉시 시크와 새 위치 기반 우선순위 재조정

5. **성능 최적화**: 적절한 버퍼링과 비동기 I/O 활용

6. **자동 관리**: 스트림 생명주기와 피어 연결 자동 관리

이러한 설계는 BitTorrent의 조각 기반 다운로드 특성을 활용하면서도 실시간 스트리밍에 필요한 성능과 사용자 경험을 제공합니다.